# VisionMate

*Your eyes, your voice, your independence | M1: Approach (due 10:00 PM, 10th October)*

## 1. Problem statement and our understanding

Blind and low-vision people in India must move, read, shop, study, travel and use machines in environments built for sighted users. **Core gap:** existing tools either give a GPS position or name an object, but they do not tell the user *what is immediately around them*, do not turn real-world text into structured speech, and do not say how far the answer can be trusted. A wrong answer about a medicine, a banknote or a road crossing is a safety risk, so reliability and honesty about uncertainty matter as much as accuracy. Language is a second barrier: most tools are English-first, while users speak Hindi, Tamil, Telugu and more, often code-mixed.

| Problem area | What goes wrong today (core challenge) | What the user needs |
|---|---|---|
| **1. Navigation and obstacles** | Head/ground hazards, potholes, open drains, crowded footpaths, road crossings, exact doorways. GPS gives location, not surroundings. | Real-time, directional, eyes-free awareness of the space and of risk. |
| **2. Reading text and documents** | Forms, menus, notices, handwriting, tables and small print stay unreadable; plain OCR reads in the wrong order. | Accurate, structure-aware speech (rows, form fields), with capture guidance. |
| **3. Identifying objects and products** | Similar packages, medicines, currency notes, colours, items lost in clutter; results change with light. | Reliable identification, a stated confidence level, and a second check for high-stakes items. |
| **4. Education accessibility** | Diagrams, graphs, equations and untagged PDFs lose information when made accessible. | Lossless description, sonified graphs, spoken navigable maths. |
| **5. Public transport, new places** | Wrong bus or platform, missed stops, unknown counters and lifts, no help when alone. | Timely, location-specific guidance and live human help. |
| **6. Daily independence, digital access** | Kiosks, ATMs, appliances, CAPTCHA and poorly labelled apps need a sighted helper. | Complete tasks alone, with the app narrating before it acts. |

## 2. Idea / abstract

**VisionMate** is a voice-first, multilingual assistive application (English, Hindi, Tamil, Telugu and more) that runs as a PWA on a laptop today and is packaged as an Android APK later. It goes beyond "point the camera and name the object" by letting the user *feel* the space, *know* how risky it is, and *know* how much to trust each answer. Six modules (SafeWalk, ReadAssist, IdentifyIt, LearnAccess, TravelMate, LifeAssist) answer the six problems, supported by VoiceControl, Mate Circle and an Emergency layer. It works with no API keys using on-device models, and improves when cloud AI is available. Privacy is built in: on-device first, no face identification, no stored images.

**Ten differentiators** (each is a first-class, testable feature):

| | |
|---|---|
| **Mate Sonar**: stereo tone; pitch and pulse rise as obstacles near, panned left/right. | **Risk Radar**: fused 0-100 score (Clear / Caution / Stop) with a spoken "Why". |
| **Mate Hazard Map**: community, anonymous, expiring hazard reports; warns before the camera sees it. | **Scene-Aware Auto Mode**: detects road, document, shelf, kiosk and switches mode itself. |
| **Spatial Memory**: "Where are my keys?" with last-seen place and time, stored locally. | **Trust Meter + Second Opinion**: High/Medium/Low confidence; stricter checks for medicine, currency, crossing. |
| **Mate Circle**: live WebRTC human helper whose arrow hints become spoken guidance. | **Dry-Run Voice Control**: narrates the plan, asks confirmation (plus PIN for payments) before acting. |
| **Route Rehearsal**: hear the full route, landmarks and hazards before leaving. | **Offline Survival Ladder**: Cloud > Hybrid > On-device > Minimal (SOS, battery, beeps), announced on change. |

## 3. High-level approach, pipeline and architecture

**Approach.** A single shared core (voice, audio, vision, context, safety, privacy) is reused by every module, so each feature only adds its own logic. Every module returns the same result type `{value, confidence, reasons[], needsSecondOpinion}`, which feeds the Trust Meter and a priority speech queue (P0 emergency, P1 hazard, P2 navigation, P3 information, P4 chatter). On-device inference is the default; the backend is only an AI proxy so keys never reach the browser.

| | |
|---|---|
| **ACTIVATION LAYER** | Wake word "Hey Mate" \| push-to-talk / Spacebar \| shake and tap gestures \| headset button \| big-button home (Walk, Read, Identify, Assist, SOS) |
| **VOICE LAYER** | STT (Web Speech, pluggable) > Intent Router (rules + LLM, code-mixed speech) > TTS with priority queue \| Machine Translation \| haptic and earcon grammar |
| **CONTEXT ENGINE** | Mode manager + Scene-Aware Auto Mode \| user profile, language, verbosity \| local personalization |
| **FEATURE MODULES A-G** | **PERCEPTION**: Object detection (TF.js / MediaPipe), MiDaS depth, optical flow, scene classifier, Sonar, Risk Radar<br>**RECOGNITION**: Tesseract.js OCR (multi-language), document and table parser, barcode, currency, colour, math, diagram reader<br>**NAVIGATION**: GPS + Leaflet/OSM, OSRM routing, indoor QR anchors, hazard map, transit, Route Rehearsal<br>**DEVICE CONTROL**: Phone Simulator, intents / deep links, Desktop Agent (allow-list, pairing, audit), Dry-Run guard |
| **SAFETY LAYER** | Confidence Engine + Trust Meter \| second-opinion check \| SOS (voice, shake, fall) \| battery guard \| offline ladder \| face blur and consent |
| **DATA + SERVICES** | IndexedDB (AES-GCM encrypted) \| Express + SQLite (hazards, circles) \| WebSocket signalling for Mate Circle and SOS \| optional LLM / Indic cloud APIs via proxy |

**Core pipelines**

| | |
|---|---|
| **Walk** | Camera frame > detector + depth + flow (adaptive 1-2 s loop) > object list (class, clock-direction, distance, height band, motion) > Risk Radar fusion with GPS speed and hazard map > Sonar tone + short spoken alert + vibration. |
| **Read** | Capture guidance (edges, glare, blur) > preprocess (deskew, contrast) > OCR > layout / table / form parsing > optional translation > TTS with reading controls. |
| **Ask / Identify** | Voice question + frame > multimodal LLM (fallback: detection + OCR + templates) > Trust Meter > second pass if high-stakes > spoken answer with confidence. |
| **Act** | Command > Intent Router > Dry-Run narration > spoken confirm (+ PIN for payments) > intent / deep link or Desktop Agent > audit log. "Stop everything" halts all automation. |

**Stack and delivery.** React 18 + Vite + TypeScript + Tailwind + Zustand (PWA); Node/Express + SQLite backend; Desktop Agent on localhost; Capacitor wrapper for Android with a Kotlin bridge (foreground service, key combos, SMS, calls). **Honest limits:** an aid, not a replacement for a cane or guide dog; Safe-Crossing gives environmental information only, never "safe to cross"; pothole detection is heuristic-first and accuracy-limited; cross-app AccessibilityService automation is not possible from a web page, so intents and deep links are used first.

## 4. Planned testing methodology

The laptop is the test device. A built-in **Test Lab** (`/testlab`) injects video, GPS, sensors, hazards and voice so every feature is testable without walking outside.

| Level | What we test and how | Pass criteria |
|---|---|---|
| **Unit (Vitest)** | Risk fusion, intent routing (incl. Hindi/Tamil/code-mixed), speech priority queue, Sonar mapping, hazard geo-queries, confidence rules. | All pass; strict TypeScript and lint clean. |
| **Integration** | Backend APIs (vision, doc, intent, hazards, circle) with zod validation, rate limits, no-key fallback response `{available:false, fallbackUsed:true}`. | Correct status codes; no image persisted. |
| **Simulated field (Test Lab)** | Walking videos and images for vision modules; GPS replay (GPX); battery, network, shake/fall and light sliders; fake hazards and obstacles at set clock-direction and distance; voice test bench showing intent and confidence. | Expected alert, mode switch or ladder rung occurs. |
| **Scenario end-to-end (Playwright)** | Scripted runs: walk to bus stop with pothole and crossing; read a form; order biryani (Phone Simulator); fall and SOS; offline degradation; Mate Circle two-window call. | Assertions pass; report generated. |
| **Accuracy and safety** | Labelled sample sets: OCR word accuracy (EN, HI, TA, TE), currency and medicine-label reads, detection precision/recall. High-stakes cases must trigger second opinion or "Ask a person"; unclear fields say "unclear", never a guess. | Accuracy reported honestly per module; zero silent wrong answers on test set. |
| **Performance** | Metrics panel: FPS, per-model latency, speech latency, memory, battery estimate. | Hazard alert < 700 ms; wake word < 1 s; OCR read start < 3 s. |
| **Accessibility and user testing** | Keyboard-only and screen-reader checks, AAA contrast, reduced motion; eyes-closed voice-only runs; early sessions with blind and low-vision users, whose feedback drives redesign. | Every screen operable by voice and keyboard. |